Services

Services are long lived pieces of code that provide a feature. They may be imported by components (with `useService`) or by other services. Also, they can declare a set of dependencies. In that sense, services are basically a DI dependency injection system. For example, the `notification` service provides a way to display a notification, or the `rpc` service is the proper way to perform a request to the Odoo server.

The following example registers a simple service that displays a notification every 5 seconds:

```
import { registry } from "@web/core/registry";

const myService = {
    dependencies: ["notification"],
    start(env, { notification }) {
        let counter = 1;
        setInterval(() => {
            notification.add(`Tick Tock ${counter++}`);
        }, 5000);
    }
};

registry.category("services").add("myService", myService);
```

At startup, the web client starts all services present in the `services` registry. Note that the name used in the registry is the name of the service.

> **Note**
>
> Most code that is not a component should be *packaged* in a service, in particular if it performs some side effect. This is very useful for testing purposes: tests can choose which services are active, so there are less chance for unwanted side effects interfering with the code being tested.

Defining a service

A service needs to implement the following interface:

odoo docs

start(*env*, *deps*)

Arguments:
env (**Environment()**) – the application environment

deps (**Object()**) – all requested dependencies

Returns:
value of service or Promise<value of service>

This is the main definition for the service. It can return either a value or a promise. In that case, the service loader simply waits for the promise to resolve to a value, which is then the value of the service.

Some services do not export any value. They may just do their work without a need to be directly called by other code. In that case, their value will be set to `null` in `env.services`.

async

Optional value. If given, it should be `true` or a list of strings.

Some services need to provide an asynchronous API. For example, the `rpc` service is an asynchronous function, or the `orm` service provides a set of functions to call the Odoo server.

In that case, it is possible for components that use a service to be destroyed before the end of an asynchronous function call. Most of the time, the asynchronous function call needs to be ignored. Doing otherwise is potentially very risky, because the underlying component is no longer active. The `async` flag is a way to do just that: it signals to the service creator that all asynchronous calls coming from components should be left pending if the component is destroyed.

Using a service

A service that depends on other services and has properly declared its `dependencies` simply receives a reference to the corresponding services in the second argument of the `start` method.

The `useService` hook is the proper way to use a service in a component. It simply returns a reference to the service value, that can then be used by the component later. For example:

```
  setup() {
    onWillStart(async () => {
      const result = await rpc(...);
    })
  }
}
```

Reference List

Technical Name	Short Description
cookie	read or modify cookies
effect	display graphical effects
http	perform low level http calls
notification	display notifications
router	manage the browser url
rpc	send requests to the server
scroller	handle clicks on anchors elements
title	read or modify the window title
user	provides some information related to the current user

Cookie service

Overview

- Technical name: `cookie`
- Dependencies: none

Provides a way to manipulate cookies. For example:

```
cookieService.setCookie("hello", "odoo");
```

odoo docs

setCookie(*name*[, *value*, *ttl*])

Arguments:
name (`string()`) – the name of the cookie that should be set

value (`any()`) – optional. If given, the cookie will be set to that value

ttl (`number()`) – optional. the time in seconds before the cookie will be deleted (default=1 year)

Sets the cookie `name` to the value `value` with a max age of `ttl`

deleteCookie(*name*)

Arguments:
name (`string()`) – name of the cookie

Deletes the cookie `name`.

Effect service

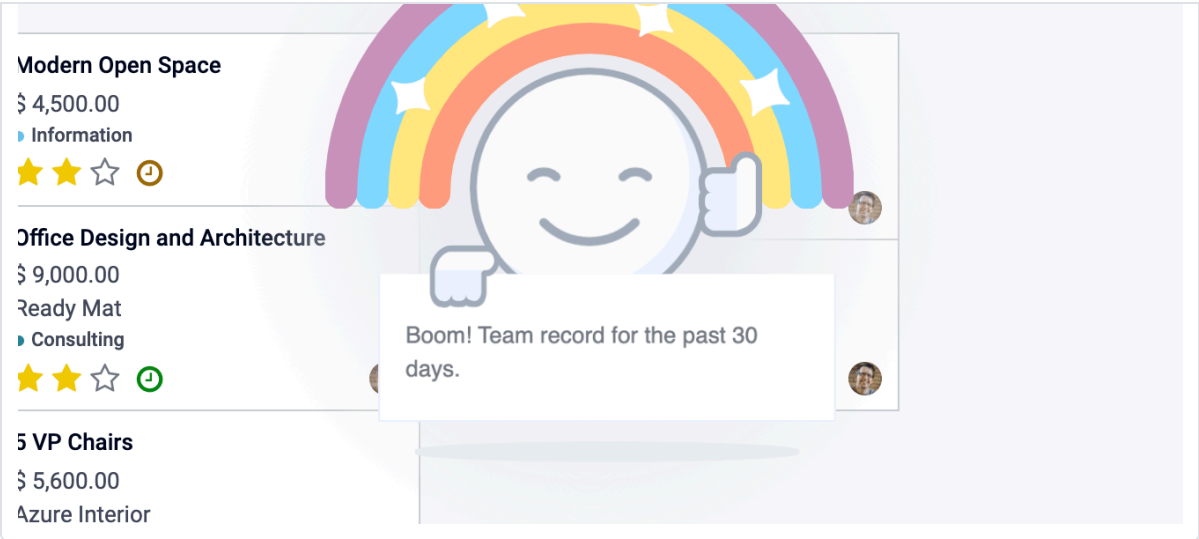
Overview

- Technical name: `effect`
- Dependencies: None

Effects are graphical elements that can be temporarily displayed on top of the page, usually to provide feedback to the user that something interesting happened.

A good example would be the rainbow man:

odoo docs



Here's how this can be displayed:

```
const effectService = useService("effect");
effectService.add({
  type: "rainbow_man", // can be omitted, default type is already "rainbow_man"
  message: "Boom! Team record for the past 30 days.",
});
```

> **Warning**
>
> The hook `useEffect` is not related to the effect service.

API

`effectService.add(options)`

Arguments:
options (`object()`) – the options for the effect. They will get passed along to the underlying effect component.

Display an effect.

The options are defined by:

```
interface EffectOptions {
  // The name of the desired effect
```

odoo docs

Available effects

Currently, the only effect is the rainbow man.

RainbowMan

```
effectService.add({ type: "rainbow_man" });
```

Name	Type	Description
params.Component	owl.Component?	Component class to instantiate inside the RainbowMan (will replace the message).
params.props	object?={}	If params.Component is given, its props can be passed with this argument.

		rainbowman holds. If effects are disabled for the user, the rainbowman won't appear and a simple notification will get displayed as a fallback. If effects are enabled and params.Component is given, params.message is not used. The message is a simple string or a string representing html (prefer using params.Component if you want interactions in the DOM).
`params.messageIsHtml`	`boolean?=false`	Set to true if the message represents html, s.t. it will be correctly inserted into the DOM.
`params.img_url`	`string? =/web/static/img/smile.svg`	The url of the image to display inside the rainbow.
`params.fadeout`	`("slow"|"medium"|"fast"|"no")? ="medium"`	Delay for rainbowman to disappear. `"fast"` will make rainbowman dissapear quickly. `"medium"` and `"slow"` will wait little longer before disappearing (can be used when `params.message` is longer).

odoo docs

anywhere outside rainbowman.

How to add an effect

The effects are stored in a registry called `effects`. You can add new effects by providing a name and a function.

```
const effectRegistry = registry.category("effects");
effectRegistry.add("rainbow_man", rainbowManEffectFunction);
```

The function must follow this API:

<newEffectFunction>(*env*, *params*)

Arguments:

env (**`Env()`**) – the environment received by the service

params (**`object()`**) – the params received from the add function on the service.

Returns:

`({Component, props} | void)` A component and its props or nothing.

This function must create a component and return it. This component is mounted inside the effect component container.

Example

Let's say we want to add an effect that add a sepia look at the page.

```
import { registry } from "@web/core/registry";
import { Component, xml } from "@odoo/owl";

class SepiaEffect extends Component {
  static template = xml`
    <div style="
        position: absolute;
        left: 0;
        top: 0;
        width: 100%;
        height: 100%;
```

odoo docs

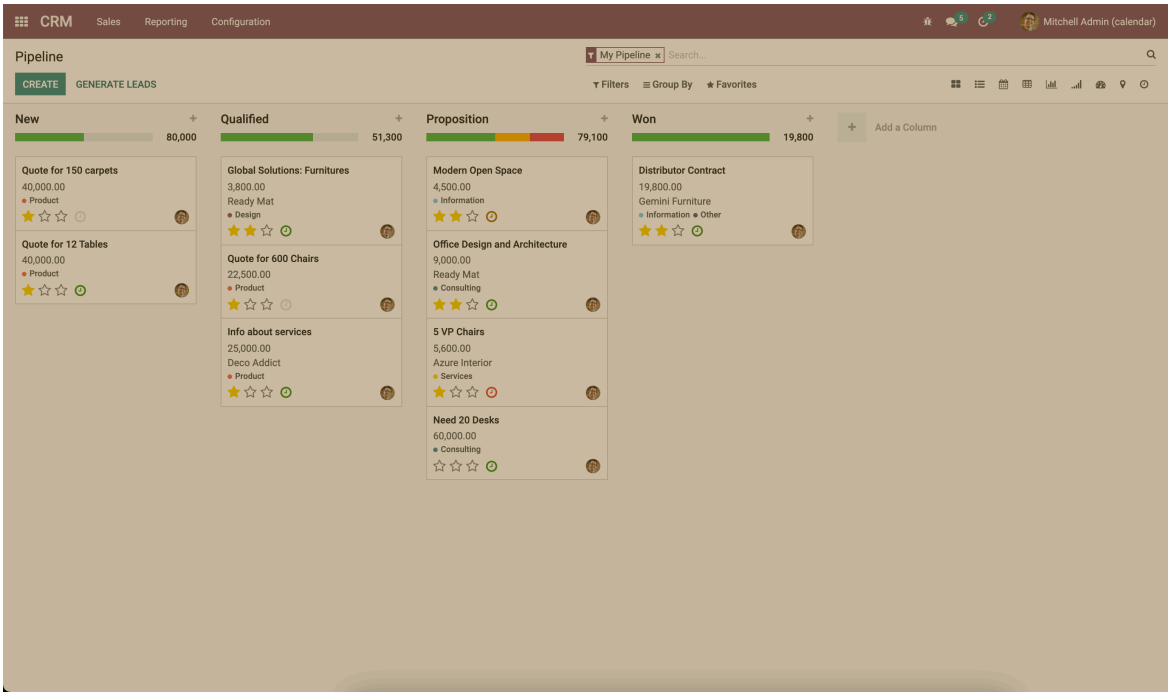
```
}

export function sepiaEffectProvider(env, params = {}) {
    return {
        Component: SepiaEffect,
    };
}

const effectRegistry = registry.category("effects");
effectRegistry.add("sepia", sepiaEffectProvider);
```

And then, call it somewhere you want and you will see the result. Here, it is called in webclient.js to make it visible everywhere for the example.

```
const effectService = useService("effect");
effectService.add({ type: "sepia" });
```



Http Service

Overview

odoo docs

), lower level control on requests may sometimes be required.

This service provides a way to send `get` and `post` **http requests**.

API

`async get(route[, readMethod = "json"])`

Arguments:
route (**`string()`**) – the url to send the request to

readMethod (**`string()`**) – the response content type. Can be “text”, “json”, “formData”, “blob”, “arrayBuffer”.

Returns:
the result of the request with the format defined by the readMethod argument.

Sends a get request.

`async post(route[, params = {}, readMethod = "json"])`

Arguments:
route (**`string()`**) – the url to send the request to

params (**`object()`**) – key value data to be set in the form data part of the request

readMethod (**`string()`**) – the response content type. Can be “text”, “json”, “formData”, “blob”, “arrayBuffer”.

Returns:
the result of the request with the format defined by the readMethod argument.

Sends a post request.

Example

```
const httpService = useService("http");
const data = await httpService.get("https://something.com/posts/1");
// ...
await httpService.post("https://something.com/posts/1", { title: "new title", conte
```

Notification service

odoo docs

The `notification` service allows to display notifications on the screen.

```
const notificationService = useService("notification");
notificationService.add("I'm a very simple notification");
```

API

add(*message*[, *options*])

Arguments:
message (**string()**) – the notification message to display

options (**object()**) – the options of the notification
Returns:
a function to close the notification

Shows a notification.

The options are defined by:

Name	Type	Description
`title`	string	Add a title to the notification
`type`	`warning` | `danger` | `success` | `info`	Changes the background color according to the type
`sticky`	boolean	Whether or not the notification should stay until dismissed
`className`	string	additional css class that will be added to the notification
`onClose`	function	callback to be executed when the notification closes
`buttons`	button[] (see below)	list of button to display in the notification
`autocloseDelay`	number	duration in milliseconds before the notification is closed automatically

odoo docs

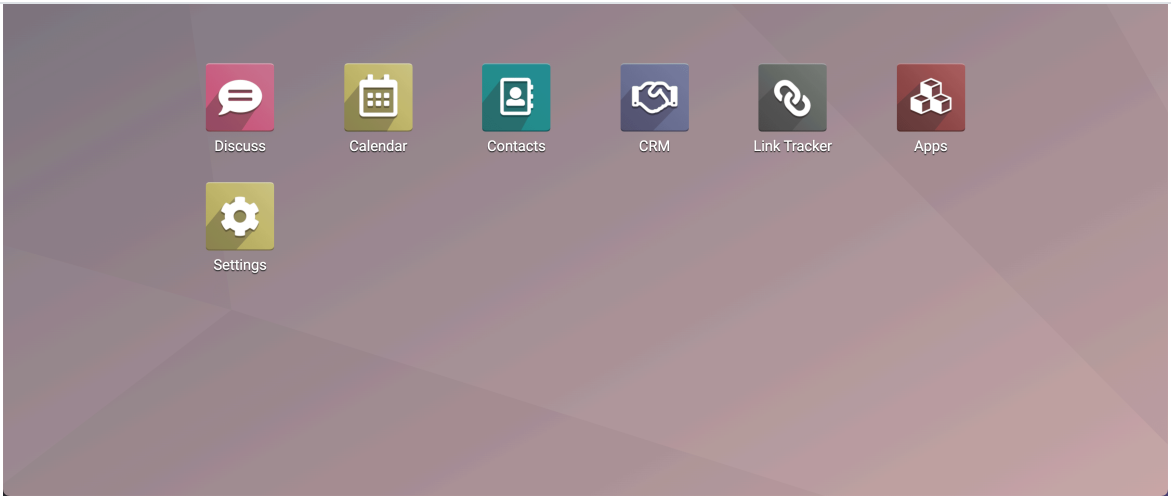
`name`	string	The button text
`onClick`	function	callback to execute when the button is clicked
`primary`	boolean	whether the button should be styled as a primary button

Examples

A notification for when a sale deal is made with a button to go some kind of commission page.

```
// in setup
this.notificationService = useService("notification");
this.actionService = useService("action");

// later
this.notificationService.add("You closed a deal!", {
  title: "Congrats",
  type: "success",
  buttons: [
      {
          name: "See your Commission",
          onClick: () => {
              this.actionService.doAction("commission_action");
          },
      },
  ],
});
```



A notification that closes after a second:

```
const notificationService = useService("notification");
const close = notificationService.add("I will be quickly closed");
setTimeout(close, 1000);
```

Router Service

Overview

- Technical name: `router`
- Dependencies: none

The `router` service provides three features:

- information about the current route
- a way for the application to update the url, depending on its state
- listens to every hash change, and notifies the rest of the application

API

`current`

The current route can be accessed with the `current` key. It is an object with the following information:

- `pathname (string)` : the path for the current location (most likely `/web`)

odoo docs

For example:

```
// url = /web?debug=assets#action=123&owl&menu_id=174
const { pathname, search, hash } = env.services.router.current;
console.log(pathname); //   /web
console.log(search); //   { debug="assets" }
console.log(hash); //   { action:123, owl: "", menu_id: 174 }
```

Updating the URL is done with the `pushState` method:

pushState(*hash: object*[, *replace?: boolean*])

Arguments:

hash (**`Object()`**) – object containing a mapping from some keys to some values

replace (**`boolean()`**) – if true, the url will be replaced, otherwise only key/value pairs from the `hash` will be updated.

Updates the URL with each key/value pair from the `hash` object. If a value is set to an empty string, the key is added to the url without any corresponding value.

If true, the `replace` argument tells the router that the url hash should be completely replaced (so values not present in the `hash` object will be removed).

This method call does not reload the page. It also does not trigger a `hashchange` event, nor a `ROUTE_CHANGE` in the **main bus**. This is because this method is intended to only updates the url. The code calling this method has the responsibility to make sure that the screen is updated as well.

For example:

```
// url = /web#action_id=123
routerService.pushState({ menu_id: 321 });
// url is now /web#action_id=123&menu_id=321
routerService.pushState({ yipyip: "" }, replace: true);
// url is now /web#yipyip
```

Finally, the `redirect` method will redirect the browser to a specified url:

odoo docs

wait (**boolean()**) – if true, wait for the server to be ready, and redirect after

Redirect the browser to `url`. This method reloads the page. The `wait` argument is rarely used: it is useful in some cases where we know that the server will be unavailable for a short duration, typically just after an addon update or install operation.

> **Note**
>
> The router service emits a `ROUTE_CHANGE` event on the **main bus** whenever the current route has changed.

RPC service

Overview

- Technical name: `rpc`
- Dependencies: none

The `rpc` service provides a single asynchronous function to send requests to the server. Calling a controller is very simple: the route should be the first argument and optionally, a `params` object can be given as a second argument.

```
import { rpc } from "@web/core/network/rpc";

 // somewhere else, in an async function:
 const result = await rpc("/my/route", { some: "value" });
```

> **Note**
>
> Note that the `rpc` service is considered a low-level service. It should only be used to interact with Odoo controllers. To work with models (which is by far the most important usecase), one should use the `orm` service instead.

API

rpc(*route*, *params*, *settings*)

odoo docs

params (`Object()`) – (optional) parameters sent to the server

settings (`Object()`) – (optional) request settings (see below)

The `settings` object can contain:

- `xhr` , which should be a `XMLHTTPRequest` object. In that case, the `rpc` method will simply use it instead of creating a new one. This is useful when one accesses advanced features of the `XMLHTTPRequest` API.
- `silent (boolean)` If set to `true` , the web client will not provide a feedback that there is a pending rpc.

The `rpc` service communicates with the server by using a `XMLHTTPRequest` object, configured to work with the `application/json` content type. So clearly the content of the request should be JSON serializable. Each request done by this service uses the `POST` http method.

Server errors actually return the response with an http code 200. But the `rpc` service will treat them as error.

Error Handling

An rpc can fail for two main reasons:

- either the odoo server returns an error (so, we call this a `server` error). In that case the http request will return with an http code 200 BUT with a response object containing an `error` key.
- or there is some other kind of network error

When a rpc fails, then:

- the promise representing the rpc is rejected, so the calling code will crash, unless it handles the situation
- an event `RPC_ERROR` is triggered on the main application bus. The event payload contains a description of the cause of the error:

  If it is a server error (the server code threw an exception). In that case the event payload will be an object with the following keys:

  - `type = 'server'`
  - `message(string)`

- `subType(string)` (optional, often used to determine the dialog title)
- `data(object)` (optional object that can contain various keys among which `debug` : the main debug information, with the call stack)

If it is a network error, then the error description is simply an object `{type: 'network'}`. When a network error occurs, a **notification** is displayed and the server is regularly contacted until it responds. The notification is closed as soon as the server responds.

Scroller service

Overview

- Technical name: `scroller`
- Dependencies: none

Whenever the user clicks on an anchor in the web client, this service automatically scrolls to the target (if appropriate).

The service adds an event listener to get `click` 's on the document. The service checks if the selector contained in its href attribute is valid to distinguish anchors and Odoo actions (e.g. `<a href="#target_element"></a>`). It does nothing if it is not the case.

An event `SCROLLER:ANCHOR_LINK_CLICKED` is triggered on the main application bus if the click seems to be targeted at an element. The event contains a custom event containing the `element` matching and its `id` as a reference. It may allow other parts to handle a behavior relative to anchors themselves. The original event is also given as it might need to be prevented. If the event is not prevented, then the user interface will scroll to the target element.

API

The following values are contained in the `anchor-link-clicked` custom event explained above.

Name	Type	Description
`element`	`HTMLElement | null`	The anchor element targeted by the href
`id`	`string`	The id contained in the href

odoo docs

> **Note**
>
> The scroller service emits a `SCROLLER:ANCHOR_LINK_CLICKED` event on the **main bus**. To avoid the default scroll behavior of the scroller service, you must use `preventDefault()` on the event given to the listener so that you can implement your own behavior correctly from the listener.

Title Service

Overview

- Technical name: `title`
- Dependencies: none

The `title` service offers a simple API that allows to read/modify the document title. For example, if the current document title is "Odoo", we can change it to "Odoo 15 - Apple" by using the following command:

```
// in some component setup method
const titleService = useService("title");

titleService.setParts({ odoo: "Odoo 15", fruit: "Apple" });
```

API

The `title` service manipulates the following interface:

```
interface Parts {
    [key: string]: string | null;
}
```

Each key represents the identity of a part of the title, and each value is the string that is displayed, or `null` if it has been removed.

Its API is:

`current`

odoo docs

`getParts()`

Returns:
Parts the current `Parts` object maintained by the title service

`setParts(parts)`

Arguments:
parts (**`Parts()`**) – object representing the required change

The `setParts` method allows to add/replace/delete several parts of the title. Delete a part (a value) is done by setting the associated key value to `null`.

Note that one can only modify a single part without affecting the other parts. For example, if the title is composed of the following parts:

```
{ odoo: "Odoo", action: "Import" }
```

with `current` value being `Odoo - Import`, then

```
setParts({
  action: null,
});
```

will change the title to `Odoo`.

User service

Overview

- Technical name: `user`
- Dependencies: `rpc`

The `user` service provides a bunch of data and a few helper functions concerning the connected user.

API

odoo docs

db	Object	Info about the database
home_action_id	(number | false)	Id of the action used as home for the user
isAdmin	boolean	Whether the user is an admin (group `base.group_erp_manager` or superuser)
isSystem	boolean	Whether the user is part of the system group (`base.group_system`)
lang	string	language used
name	string	Name of the user
partnerId	number	Id of the partner instance of the user
tz	string	The timezone of the user
userId	number	Id of the user
userName	string	Alternative nick name of the user

`updateContext(update)`

Arguments:
update (**`object()`**) – the object to update the context with

update the **user context** with the given object.

```
userService.updateContext({ isFriend: true })
```

`removeFromContext(key)`

Arguments:
key (**`string()`**) – the key of the targeted attribute

remove the value with the given key from the **user context**

```
userService.removeFromContext("isFriend")
```

odoo docs

Returns:
`Promise<boolean>` is the user in the group

check if the user is part of a group

```
const isInSalesGroup = await userService.hasGroup("sale.group_sales")
```

Get Help

Contact Support | Ask the Odoo Community